Partition Algorithm

How would you solve the following problem?

Given an array of n unsorted integers, select a random element from the array and indicate the position where that element should be located if the array would be sorted. Assume, all array elements are distinct.

data = [10, **3**, **6**, 9, **2**, **4**, 15, 23]

randomElement= 6

Output: 3

Partition Algorithm - Objective

It is a technique used to divide a collection of data (usually an array) into two distinct parts based on a chosen "pivot" element, placing all elements smaller than the pivot on one side and all elements larger than the pivot on the other side. The pivot ends up in its correct position.

Original array:

			pivot				
65	86	100	**90**	95	72	70	92

Swap pivot, so it is at index 0:

pivot							
90	86	100	65	95	72	70	92

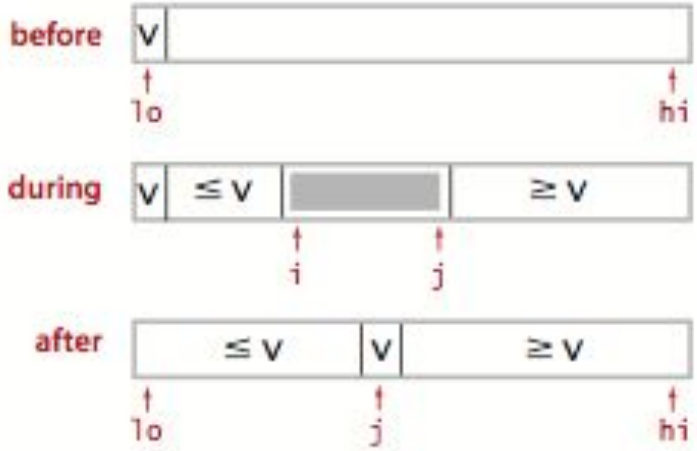
Applying the partition algorithm:

				pivot			
72	86	70	65	**90**	95	100	92
	<=90					>=90	

Partition Algorithm - Strategy

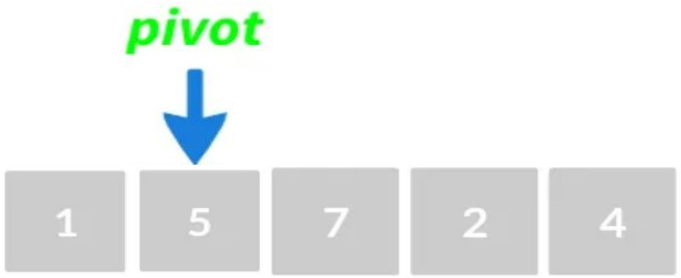
Choose an element from the array, we are going to call it pivot.

Then, create partitions based on the pivot



Partition Method: Step-by-Step

1. Create an array of integers and randomly choose an index from the array. The value at that index will be your pivot.

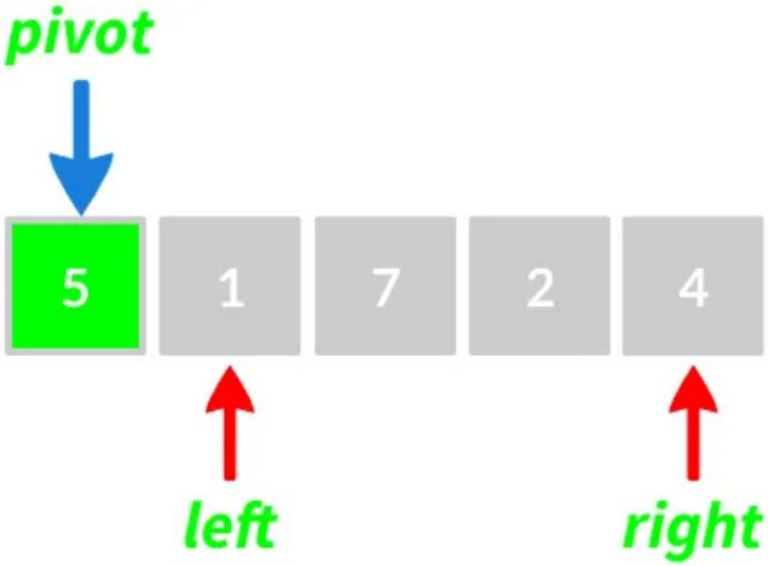


2. If the selected index is not at index zero Swap the pivot with the element at index zero.

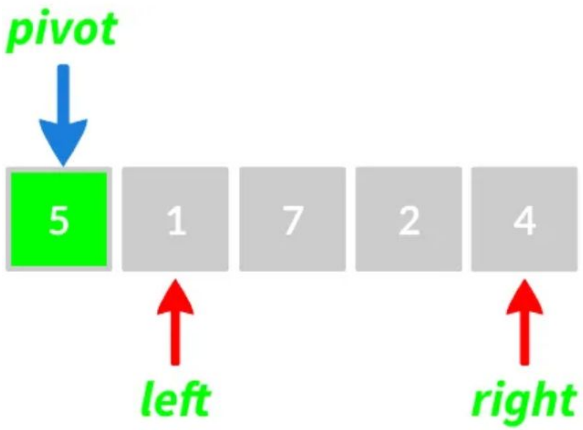
Partition Method: Step by Step

3. Set up "left" and "right" pointers:

“left index” => “leftmost” element
“right index” => "rightmost” element



Partition Method: Step-by-Step



Pair activity:

Follow this rules and find the next steps.

Iterate over the array until left crosses right:

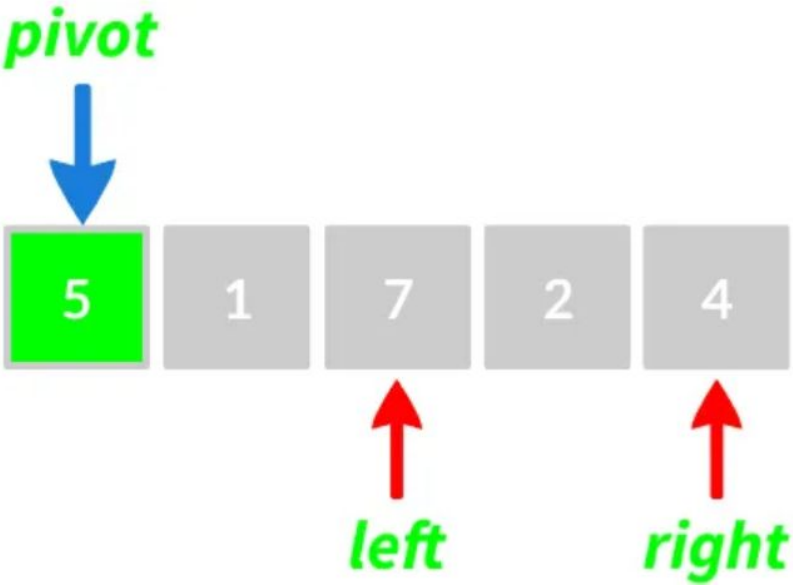
- While arr[left] ≤ pivot, move left forward
- While arr[left] > pivot, swap arr[left] with arr[right] and move right backward

Once left ≥ right, swap the pivot (arr[0]) with arr[left].

Use your notebook or whiteboard, draw the status of the array and indices (left, right) after each iteration.

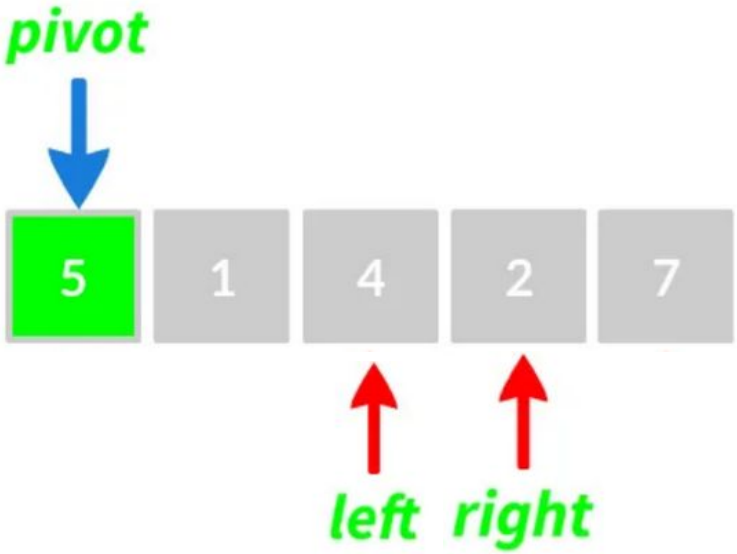
Partition Method: Step by Step

4. Move left cursor to the right



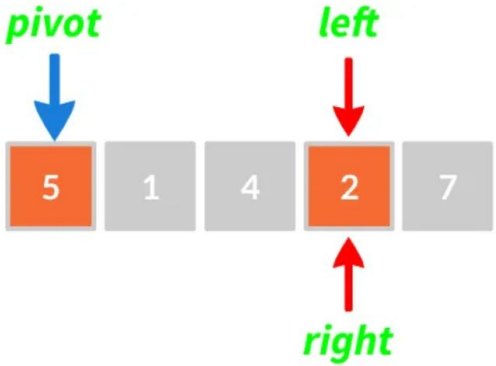
Partition Method: Step by Step

5. Swap between “right” and “left” elements and move right cursor to the left.

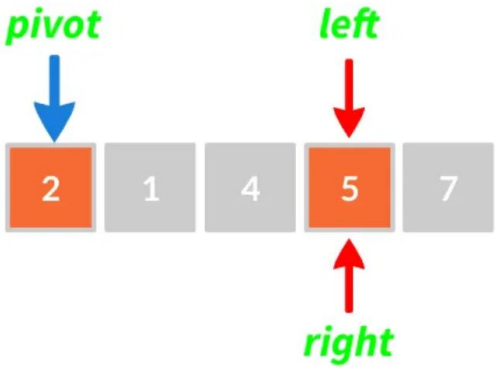


Partition Method: Step by Step

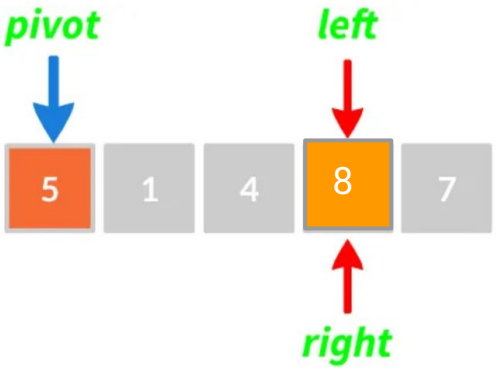
6. Move left cursor to the right.



7. Swap the left element with the pivot



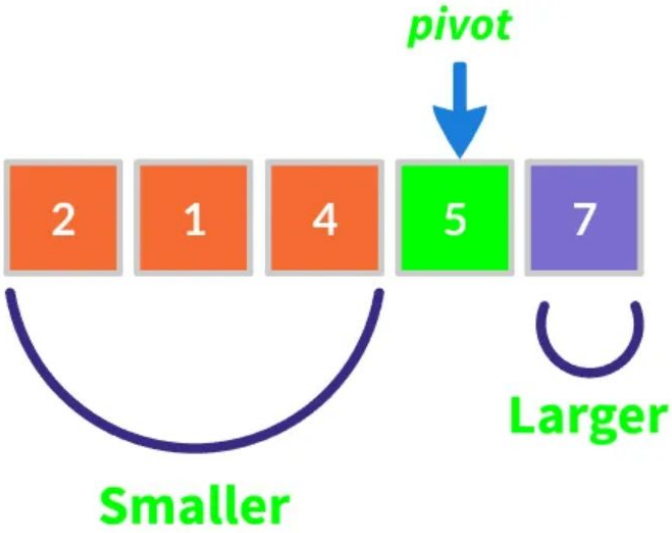
What if the element arr[left] is greater than the pivot?



Swap the element at **arr[left - 1]** with the pivot

Partition Method: Step-by-Step

8. Return the index of the **final position of the pivot element => 3**



Coding Time!!!!

Write your code here:
../APCSA/apcsa-assignments-spring-YourUsername/classwork/03_02_partition/Partition.java

You must implement the following method:

```
public static int partition( int [] data, int start, int end){
    // You must declare variables to track left and right indices
}
```

Call the method like this: `partition(yourArray, 0, yourArray.length - 1));`

Note: start and **end** define the **boundaries of the array**, but you need **left** and **right** as **moving pointers** that walk toward each other during the partitioning process. **start** and **end** need to stay unchanged so you know the original bounds, while **left** and **right** do the actual work of scanning and swapping elements around the pivot.